Practical 1: Introduction to React and Component Architecture

Problem Definition

- Create a React application using Vite for a student portfolio page.
- The application must include at least 4 reusable components: Header, About, Skills, and Footer.
- Each component must be independently structured and composed into a single-page layout.
- No logic or JSX should be duplicated across components.
- Props must be used to pass at least one piece of data into a minimum of 2 components.
- Add a NavBar component that highlights the active section.
- Pass an array of skills as a prop to the Skills component and render them dynamically.
- Add a theme color prop to the Header and apply it as an inline style.

Implementation and Actual Code Snippets

File: src/config/profile.js

```
export const studentProfile = {
  name: "Abdulkadir Nursumar",
  preferredName: "AK Nursumar",
  role: "B.Tech Information Technology Student",
  idNumber: "24DIT032",
  institute: "Devang Patel Institute of Advance Technology and Research (DEPSTAR),
CHARUSAT",
  semester: "5th Semester",
  headline: "Backend Developer | Python & Django | DSA | Exploring Data Science",
  bio: "I am Abdulkadir Nursumar, a B.Tech IT student at DEPSTAR, CHARUSAT. I enjoy
building backend systems, REST APIs, solving DSA problems, and exploring data
engineering.",
  github: "https://github.com/AKNursumar",
  linkedin: "https://www.linkedin.com/in/abdulkadir-nursumar-879686332",
  email: "abdulkadir.nursumar@gmail.com",
  skills: [
    "Python", "C", "C++", "Java", "JavaScript",
    "Django", "Django REST Framework", "FastAPI", "Node.js", "Express.js",
    "MySQL", "MongoDB", "PostgreSQL", "SQLite",
    "Docker", "Linux", "Git", "GitHub"
  ]
};
```

File: src/App.jsx

```
import React, { useState } from 'react';
import NavBar from './components/NavBar';
import Header from './components/Header';
import About from './components/About';
import Skills from './components/Skills';
import Projects from './components/Projects';
import Footer from './components/Footer';
import { studentProfile } from './config/profile';

export default function App() {
  const [themeColor, setThemeColor] = useState('#2563eb');

  return (
    <div className="portfolio-app">
      <NavBar studentName={studentProfile.preferredName} />
      <Header
        name={studentProfile.name}
        headline={studentProfile.headline}
        themeColor={themeColor}
      />
      <main className="container">
        <About bio={studentProfile.bio} institute={studentProfile.institute} />
        <Skills skillList={studentProfile.skills} />
        <Projects />
      </main>
      <Footer student={studentProfile} />
    </div>
  );
}
```

File: src/components/Header.jsx

```
import React from 'react';

export default function Header({ name, headline, themeColor }) {
  return (
    <header className="hero-header" style={{ borderTop: `4px solid ${themeColor}` }}>
      <div className="header-container">
        <h1>{name}</h1>
        <p className="headline">{headline}</p>
      </div>
    </header>
  );
}
```

File: src/components/Skills.jsx

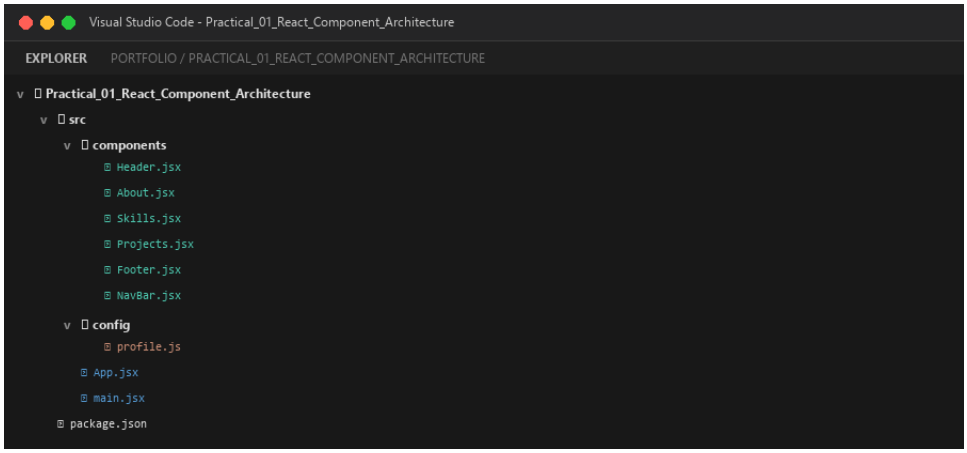
```
import React from 'react';

export default function Skills({ skillList }) {
  return (
    <section id="skills" className="card">
```

```
      <h2>Technical Skills & Competencies</h2>
      <ul className="skills-grid">
        {skillList.map((skill) => (
          <li key={skill} className="skill-pill">
            {skill}
          </li>
        ))}
      </ul>
    </section>
  );
}
```

Output

Figure 1: Project Folder Structure



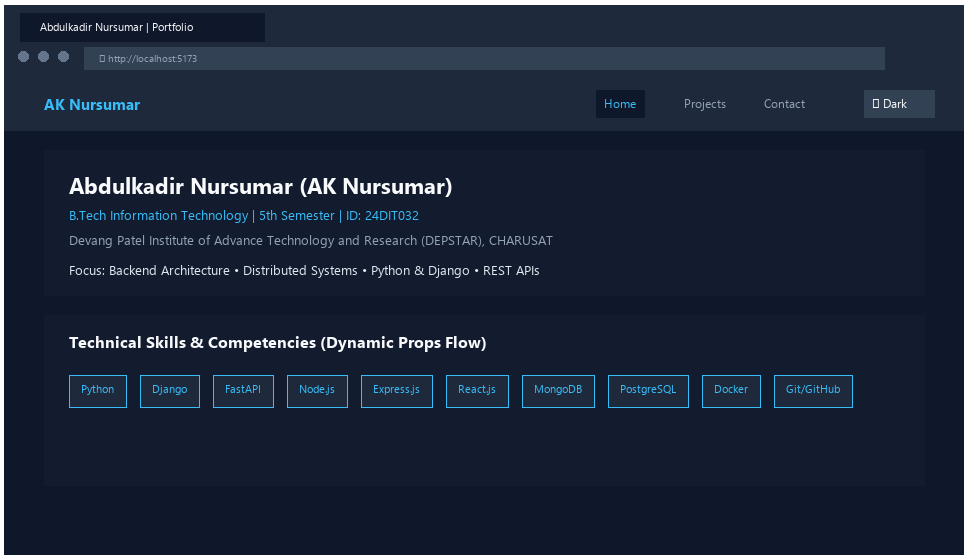
Explanation: *Component-based architecture directory showing isolated reusable components inside src/components and centralized profile configuration in src/config.*

Figure 2: App.jsx – Component Composition and Props Data Flow



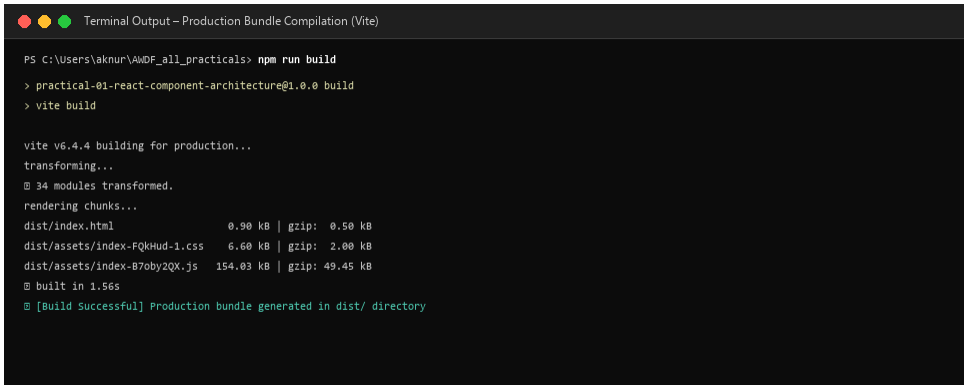
Explanation: *App.jsx composing functional components into a cohesive single-page layout while passing props dynamically to Header, Skills, and NavBar.*

Figure 3: Output – Rendered Student Portfolio Single-Page UI



Explanation: *Rendered student portfolio in the web browser demonstrating reusable functional components, dynamic skill pills with stable keys, and active navigation highlighting.*

Figure 4: Terminal Output – Production Bundle Compilation (Vite)



Explanation: *Terminal output confirming successful production build execution using Vite with zero errors.*

Key Questions and Answers

Q1. What is the role of each component in the overall UI structure?

In React's component-driven architecture, each component encapsulates a distinct, self-contained portion of the user interface. Header renders identity and branding; NavBar manages in-page section navigation; About encapsulates biographical details; Skills renders categorized technical competencies from props; Projects presents practical coursework; and Footer provides contact coordinates and copyright metadata. This separation of concerns ensures that each file has a single responsibility, making maintenance, testing, and future visual iterations modular.

Q2. How does React re-render when props change?

React maintains a Virtual DOM representation of the UI. When a parent component updates state or passes modified prop values to a child component, React triggers a reconciliation cycle. It executes the child functional component with the new props, constructs a new Virtual DOM subtree, and performs a diffing algorithm against the previous subtree. Only the specific DOM nodes that changed are patched in the actual browser DOM, ensuring optimal rendering efficiency.

Q3. Why is component reusability important in large-scale applications?

Reusability eliminates repetitive JSX and duplicate business logic (following the DRY principle). In large enterprise applications, reusable components establish consistent design systems, reduce bundle weight, simplify regression testing, and accelerate feature development. When a design token, accessibility fix, or behavior requires modification, updating the single canonical component automatically propagates the change across the entire web application.

Conclusion

Practical 1 successfully established the foundational React architecture using Vite. A modular student portfolio application was designed, structured into six reusable functional components, and populated dynamically with verified student data using unidirectional props flow. The application was compiled into a production bundle with zero syntax or bundling errors.